Assignment: Job Skills Annotation Tool

Career Intelligence Platform — Data Labelling Track

Difficulty: Intermediate Python | **Points:** 50 | **Deadline:** 5 days from receipt

---

Background

We have scraped and normalized **100 job descriptions** from 13 top companies (Accenture, Apple, Google, Cognizant, Mastercard, and others). The raw JD text is stored in `jobs_annotation_dataset.csv`. Each row already has auto-extracted skill fields (`skills_required`, `skills_preferred`) from our scraper pipeline — but these are noisy and incomplete.

Your job is to build a **Python desktop tool** that:

1. Shows a human annotator each JD one at a time
2. Parses and surfaces skills from the raw text automatically
3. Takes human input for validated skills and notes
4. Writes the annotations back into the CSV under new columns
5. Runs until all 100 rows are annotated (with skip/resume support)

This tool will directly power our skills taxonomy — the annotations you produce here become ground truth for model training.

Files You Are Given

File	Description
`jobs_annotation_dataset.csv`	100 jobs, 13 companies, max 10 per company
`job_annotator.py`	Reference implementation (study this, then extend it)

CSV Schema

Column	Type	Description
`job_id`	str	Unique identifier
`title`	str	Job title
`company_name`	str	Employer
`raw_jd_text`	str	Full JD as scraped
`skills_required`	str	Auto-extracted required skills (Python list as string)
`skills_preferred`	str	Auto-extracted preferred skills
`seniority_level`	str	junior / mid / senior
`location_city`	str	—
`min_years_experience`	float	—
`max_years_experience`	float	—
`human_skills_required`	str	**YOUR OUTPUT — comma-separated**
`human_skills_good_to_have`	str	**YOUR OUTPUT — comma-separated**
`human_notes`	str	**YOUR OUTPUT — freetext**
`annotated`	str	`'True'` / `'False'` — set to `'True'` on submit

Phase 1 — Run the Reference Tool *(no points, mandatory)*

Before writing any code:

1. Install dependencies: `pip install pandas`
2. Place `job_annotator.py` and `jobs_annotation_dataset.csv` in the same folder
3. Run: `python job_annotator.py`
4. Annotate at least **5 rows** yourself to understand the workflow
5. Open the CSV and confirm those 5 rows now have `annotated = True` with your inputs

This gives you hands-on understanding of what you are building before you extend it.

Phase 2 — Improve the Skill Parser *(20 points)*

The reference tool uses a static keyword list to auto-detect skills from JD text. Your task is to make it smarter.

Deliverable: `skill_parser.py`

Requirements:

Implement a function `extract_skills(jd_text: str) -> dict` that returns:
{ "required": ["python", "sql", ...], "good_to_have": ["tableau", "spark", ...], "experience_years": 3, # extracted number or None "seniority": "mid" # junior/mid/senior or None}

-
- The `required` vs `good_to_have` split must use JD section signals — look for phrases like "Must have", "Required", "Minimum Qualifications" vs "Preferred", "Good to have", "Nice to have", "Bonus"
- Skill matching must be case-insensitive and handle multi-word skills (e.g., "machine learning", "power bi", "ci/cd")
- Write **10+ pytest test cases** in `test_skill_parser.py` covering:
    - JD with clear required/preferred sections
    - JD with no skills mentioned
    - JD with typos or alternate spellings (e.g., "PostgresQL", "Node JS")
    - JD mixing technical and soft skills

Scoring:

- Correct required/preferred split logic: 10 pts
- Test coverage and edge cases: 6 pts
- Code clarity and docstrings: 4 pts

Phase 3 — Extend the UI *(20 points)*

Extend `job_annotator.py` with the following features.

Feature A — Skill chips (8 pts)

When the parser detects skills, display them as clickable chip buttons rather than plain text. Clicking a chip in the "Auto-detected" strip adds it to the "Must-have" or "Good-to-have" text box (your choice of which). This removes the need to type common skills by hand.

Feature B — Progress persistence across sessions (6 pts)

If the annotator closes the tool mid-session and reopens it, it must resume from where it left off — skipping already-annotated rows and showing the correct progress count. This should already mostly work via the `annotated` column, but you need to verify it is robust:

- Test that abruptly killing the process (Ctrl+C) does not corrupt the CSV
- Implement a write pattern that saves atomically (write to a temp file, then rename)

Feature C — Summary dashboard (6 pts)

After all 100 rows are annotated, show a summary screen that displays:

- Total annotations completed
- Top 10 most-annotated must-have skills (bar chart using matplotlib or tkinter Canvas)
- Top 5 companies by average skills count
- A "Export to CSV" button that saves a clean skills-only export: `annotations_summary.csv` with columns: `job_id`, `company_name`, `title`, `human_skills_required`, `human_skills_good_to_have`

Phase 4 — Database Integration *(10 points)*

Store annotations in a **SQLite database** alongside the CSV, so the data is queryable beyond flat files.

Deliverable: `db_writer.py`

Schema to implement:

```
CREATE TABLE job_annotations (
    job_id          TEXT PRIMARY KEY,
    company_name    TEXT,
    title           TEXT,
    seniority_level TEXT,
    skills_required TEXT,   -- JSON array string
    skills_preferred TEXT,  -- JSON array string
    human_required  TEXT,   -- JSON array string
    human_good_to_have TEXT,
    human_notes     TEXT,
```

```
    annotated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Requirements:

- On each submit in the UI, write to SQLite **and** the CSV (both must stay in sync)
- Write a standalone script `query_annotations.py` that prints:
    - All annotated jobs from `company_name = 'Google'`
    - The 5 most frequently appearing human-annotated required skills across all rows

---

Submission Checklist

- [ ] `skill_parser.py` with `extract_skills()` function
- [ ] `test_skill_parser.py` with 10+ passing tests
- [ ] `job_annotator.py` (extended with Phase 3 features)
- [ ] `db_writer.py` with `query_annotations.py`
- [ ] `jobs_annotation_dataset.csv` — with all 100 rows annotated (`annotated = True`)
- [ ] `README.md` — setup instructions, how to run each script, known issues

Submit as a ZIP or a private GitHub repo link.

Evaluation Rubric

Phase	Max Points	Key Criteria
Phase 2 — Parser	20	Required/preferred split accuracy, test coverage
Phase 3 — UI	20	Chip UI usability, atomic save, summary charts
Phase 4 — Database	10	Schema correctness, sync with CSV, query script
Code quality	—	Naming, comments, no dead code, no hardcoded paths

Total: 50 points

Tips

- Read the reference `job_annotator.py` before writing a single line — understand how it loads, navigates, and saves before extending it.
- The `raw_jd_text` column contains the full scraped HTML/text. It can be noisy — look for structural patterns (line breaks, colons, bullet markers) to find skill sections.
- SQLite comes bundled with Python — no install needed. Use `import sqlite3`.

For atomic CSV writes:
import tempfile, os, shutilwith tempfile.NamedTemporaryFile('w', delete=False, suffix='.csv') as tmp: df.to_csv(tmp.name, index=False)shutil.move(tmp.name, DATASET_PATH)

-
- If tkinter is not available on your system: `sudo apt install python3-tk` (Linux) or it ships by default on macOS and Windows.

Questions? Ping on internshala.